

Season-Aware Procedural Tree Animation with L-System Growth, Shadow Mapping, and Wind Response

Fundamentals of Computer Graphics (IGR202)

FATEMEH SHIRVANI*, Institut Polytechnique de Paris, France



Fig. 1. Teaser montage showing seasonal progression, canopy variation, and wind-driven motion.

This report presents an interactive real-time procedural tree system in C++/OpenGL that integrates geometry generation, seasonal simulation, and shadowed raster rendering. Tree structure is generated from a stochastic L-system with canopy guidance, while branch and leaf meshes are regenerated progressively under staged growth activation. Seasonal behavior is modeled as a continuous four-phase cycle (Spring–Winter) with smooth interpolation and explicit leaf-lifecycle control, including detachment, airborne transport, landing, and ground decay. Leaf appearance uses multi-variant atlases with alpha-tested cutouts and deterministic per-leaf color variation to avoid uniform autumn canopies. Wind response combines filtered direction/gust dynamics, hierarchy-dependent branch bending through damped oscillators, and coupled leaf flutter that preserves branch-attachment cues. A two-pass shadow-mapping pipeline with consistent alpha testing in depth and shading passes maintains foliage silhouette coherence. The resulting system supports runtime interaction for growth, seasons, wind, subdivision, and capture while reducing temporal artifacts such as popping, pumping, and discontinuous leaf-state transitions.

Additional Key Words and Phrases: procedural tree modeling, L-system, seasonal simulation, wind animation, real-time rendering, shadow mapping

1 Introduction

Natural vegetation is a central visual component in many graphics applications, but producing convincing trees remains difficult because structure, motion, and appearance are strongly coupled across scales. A credible tree model requires biologically plausible branching, coherent leaf distribution, physically plausible wind response, and seasonally consistent material evolution. Each component is challenging in isolation, and coupling all components introduces additional temporal failure modes such as popping, pumping, and mesh instability.

*Instructors: Amal Dev PARAKKAT and Kiwon UM

Author's address: Fatemeh Shirvani, fatemeh.shirvani@ip-paris.fr, Institut Polytechnique de Paris, Palaiseau, France.

This project develops a real-time procedural tree renderer and animator to address that integration problem. The core idea is to keep a deterministic leaf identity and branch hierarchy while allowing each subsystem (growth, season, wind, and shedding) to operate through continuous parameter updates. The implementation is written in modern OpenGL with a CPU-side mesh synthesis stage and shader-based textured rendering.

The practical objectives were:

- Generate a natural-looking branching structure from an L-system and incremental growth stages.
- Place leaves in a non-uniform yet controlled canopy distribution with reduced overlaps.
- Implement four seasons (Spring, Summer, Fall, Winter) with smooth transitions in density, size, color, and texture blending.
- Simulate leaf detachment and falling so that leaves disappearing from branches are the same leaves seen in flight and on the ground.
- Add wind that bends branches and moves leaves naturally, with stronger response near upper canopy regions.
- Maintain real-time interaction and shadowed rendering quality.

The system is interactive and controlled by keyboard shortcuts. The most important keys are:

- G: unlock next L-system growth stage.
- C: advance to next season.
- W: toggle wind physics.
- R: restart plant growth from a new random seed.



Fig. 2. Growth progression under repeated G key presses: trunk emergence, branching expansion, and leaf appearance.

2 System Scope and Contribution Focus

2.1 Integrated Module Overview

The implemented system combines three tightly coupled components:

- **Geometry:** procedural branch topology generation from L-system rules, branch mesh synthesis as tapered cylinders, and atlas-based leaf quad generation with per-leaf orientation and collision-aware placement.
- **Simulation:** staged branch growth, seasonal state transitions, leaf attachment/detachment lifecycle, airborne and landed fallen-leaf dynamics, and runtime wind response with hierarchy-dependent behavior.
- **Rendering:** textured bark and foliage shading, alpha-tested leaf rendering, multi-light shadow mapping pipeline, and seasonal color blending.

2.2 Contribution Focus

The main contribution is **simulation coherence**. Geometry and rendering were already functional, but making all dynamic systems work together without visual instability was difficult.

Most artifacts came from interactions across modules. During seasonal transitions, leaves could be added or removed in one frame, causing popping. Re-running overlap resolution on moving targets could shift leaf positions back and forth, causing canopy pumping. Wind introduced extra instability when branch motion and leaf motion were not synchronized, creating double movement or jitter. Growth-stage updates also changed branch geometry, which could desynchronize attached-leaf states.

To reduce these problems, deterministic leaf IDs, persistent per-leaf state, and smoothed leaf anchors were introduced across mesh rebuilds. Leaf placement remained collision-aware but was stabilized with target-space persistence. Seasonal transitions were staged (mature, detach, clear, regrow) to avoid abrupt state flips. Wind forcing was filtered and applied with hierarchy-dependent amplitudes so branch bending and leaf flutter stayed coherent.

Table 1. Module integration overview and where each component is discussed.

Module	Implemented elements	Detailed discussion
Geometry	L-system branch generation, branch cylinder meshing, leaf placement and atlas mapping	Sections 4.2–4.3 and implementation issue chronology
Simulation	Seasonal transitions, leaf lifecycle, wind dynamics, temporal smoothing/stability controls	Sections 4.4–4.6, 5, and 6
Rendering	Shadow-map pipeline, alpha-tested foliage, season-aware color/texture blending	Section 4.7 and teaser/results figures

2.3 Foundations from TP1 and TP2

This project extends two earlier TPs.

TP1 (TPShadowMapping) provided the rendering backbone: a dedicated depth FBO per light, light-space matrices built with `glm::ortho` and `glm::lookAt`, and a strict two-pass pipeline (depth pass then shaded pass). This structure was retained and adapted to dense foliage by applying consistent alpha testing in both passes and by preserving shadow-map debug export.

TP2 (TPSubdivisionSurface) provided the mesh-processing backbone: edge-based odd-vertex creation, even-vertex updates from adjacency, and explicit boundary rules. The `subdivideLoop()` operator was retained as an interactive refinement tool and integrated into the tree workflow while maintaining consistent normals and UV updates after refinement.

Because TP1 and TP2 already solved core rendering and mesh-processing tasks, this project could focus on the harder integration problems: growth, seasonal transitions, detachment logic, fallen-leaf lifecycle, and wind-driven motion.

3 Related Works

3.1 Procedural Structure Modeling

Procedural modeling of plants is classically rooted in formal grammars and recursive growth systems [1]. L-systems remain attractive in real-time contexts because they encode complex branching topology with compact symbolic rules and a small set of tunable parameters. However, pure grammar expansion often produces visually repetitive crowns if no additional guidance is provided.

To improve shape realism and control, attractor-based approaches such as space colonization introduce a geometric competition process between growth tips and spatially distributed attractors [2]. This allows explicit control of crown silhouette, empty regions, and branch spread. Complementary self-organizing models emphasize bud/branch competition and internal signaling to produce realistic crown development under local resource competition [3]. In this

project, L-systems were not fully replaced by colonization; however, the same design principle was adopted: separate topological generation from spatial guidance and canopy shaping.

3.2 Inverse and Data-Driven Procedural Modeling

Recent work addresses the inverse problem: instead of manually designing rules, infer a procedural representation from examples. Stava et al. presented automatic generation of parametric L-systems from 2D structures by clustering repeated transformed elements [6]. Later, inverse procedural modeling was extended to stochastic 3D trees and parameter estimation with optimization [7].

More recent methods push this direction with learning-based inference. Guo et al. infer compact L-system descriptions from branching images [8], while Lee et al. propose a transformer-based latent L-system model that generates plausible tree strings directly from learned distributions [9]. These works are beyond the implementation scope of this project, but they strongly motivate two engineering decisions: (1) keeping branch state deterministic and parameterized, and (2) exposing meaningful controls that can be optimized or learned later.

3.3 Wind Animation for Trees

Wind animation methods span from simple periodic sway to physically guided branch dynamics. Pure sine-based approaches are computationally cheap but often produce unrealistic phase-locked canopy motion. Habel et al. propose physically guided real-time animation combining branch deformation modeling and frequency-domain wind response synthesis [5]. Derzaph and Hamilton model both immediate motion and long-term growth adaptation under wind exposure [4]. Yildiz reports a practical hybrid strategy combining spring-force behavior and stochastic oscillation for real-time tree motion [10].

The final wind model in this project is inspired by this spectrum: rather than full finite-element simulation, filtered stochastic forcing, depth/radius-dependent flexibility, and damped branch response are used. This gives plausible phase lag and hierarchy-dependent motion while remaining interactive.

3.4 Foliage Rendering and Shadows

For real-time rendering, shadow mapping remains a dominant practical method [11]. Foliage introduces alpha-cutout materials, where the consistency between depth and shading passes is essential to avoid card-shaped shadows. Classical particle and textured geometry techniques for vegetation and foliage also highlight the trade-off between visual richness and efficiency [12]. The renderer follows this pragmatic strategy: alpha-tested quads with consistent thresholding in both shadow and main passes, combined with tuned shadow bias/filtering to reduce flicker.

3.5 Positioning of This Project

Compared with prior work, this project is not proposing a new standalone algorithm. Its contribution is an integration-focused pipeline where procedural growth, seasonal state transitions, wind response, and leaf lifecycle simulation are coupled with explicit temporal-coherence safeguards. The implementation emphasis is therefore on “failure-resistant” behavior in an interactive system:

Table 2. Literature survey summary and influence on implementation choices.

Reference	Main contribution	Influence on this project
Prusinkiewicz and Lindenmayer [1]	Grammar-based plant modeling framework	L-system-driven staged branch generation and compact parameter controls
Runions et al. [2] and Palubicki et al. [3]	Space colonization and self-organizing tree growth under local competition	Canopy guidance mindset, apical control intuition, and competition-driven shape variation
Stava et al. [6, 7]	Inverse procedural/L-system parameter inference	Deterministic branch state and interpretable tuning parameters
Guo et al. [8] and Lee et al. [9]	Learning-based inverse and latent L-system generation	Motivation for future automation of rule tuning from examples
Habel et al. [5], Derzaph and Hamilton [4]	Real-time physically guided and growth-aware wind behavior	Filtered gust dynamics, hierarchy-dependent flexibility, damped response
Williams [11] and Reeves/Blau [12]	Real-time shadowing and efficient foliage representation	Alpha-tested leaf quads with consistent shadow/main-pass cutouts

avoiding popping during transitions, preserving leaf identity across attach/detach states, and preventing wind artifacts such as rigid synchronization or structural instability.

4 Technical Details

4.1 System Architecture

The codebase is organized around three major components:

- **Plant state:** branch hierarchy, leaf state maps, and fallen-leaf particle-like elements.
- **Scene state:** season controller, shader/material bindings, and rendering passes.
- **Interaction loop:** input callbacks, per-frame updates, and draw calls.

The executable is built with CMake and links GLFW, GLAD, and GLM. The main loop updates season parameters, growth progression, wind simulation, and fallen leaves, then runs shadow and main render passes.

4.2 Procedural Tree Growth

Each branch stores start/end positions, growth direction, depth, radius, age, parent index, parent attachment coordinate, and `activationStage`. The `activationStage` field is central to staged growth. Topology is generated first, then only branches with `stage ≤ current growth`

Algorithm 1 Procedural Growth Compilation (L-system and Legacy-Equivalent Paths)

-
- 1: **if** legacy-equivalent mode is enabled **then**
 - 2: build topology by recursive child spawning with stochastic parameters
 - 3: assign activation stages from parent stage and branch depth
 - 4: **else**
 - 5: rewrite ω for N iterations using stochastic productions
 - 6: interpret ω with a 3D turtle and stack
 - 7: initialize branch ages/endpoints for staged activation
-

stage are allowed to initiate while parent branches continue to evolve.

In the L-system sense of Prusinkiewicz and Lindenmayer [1], a rewriting alphabet is used:

$$V = \{A, F, +, -, \&, ^, /, \backslash, [,],\},$$

where A is an active apex symbol, F emits a branch segment, rotation symbols modify turtle orientation, and $[]$ push/pop turtle state for branching. Sentence expansion is stochastic:

$$\omega_{k+1} = P_k(\omega_k), \quad \omega_0 = A, \quad (1)$$

with k the rewrite iteration. The continuation probability for $A \rightarrow FA$ is depth-stage controlled:

$$p_{\text{trunk}}(k) = \text{clamp}(((1 - s_k)0.90 + s_k 0.28)a + 0.10, 0.18, 0.92),$$

$$s_k = \frac{k}{N - 1}, \quad (2)$$

with a the apical-dominance parameter, following the same biological intuition of dominant-vs-lateral growth competition used in self-organizing tree models [3]. When continuation is not selected, a split production is sampled, e.g. $F[+A][-A]A$, $F[+A][-A][\&A]A$, or $F[+A][-A][\&A][^A][\backslash A]A$. For example, one sampled trajectory can be

$A \Rightarrow F[+A][-A]A \Rightarrow F[+F[+A][-A]A][-F[+A][-A]A]F[+A][-A]A$,

which illustrates how stochastic branching rapidly increases structural diversity.

In the current project configuration, `useLegacyEquivalentLSystem` is enabled by default, so generation uses a legacy-equivalent mode. This does not disable L-system ideas; instead, it keeps the same staged-growth semantics and stochastic branching behavior while constructing topology through the established branch-spawn kernel. In this mode, each branch spawns children with randomized count, azimuth, tilt, attachment coordinate, and geometric variation, then activation stages are assigned by parent/depth logic. This mode was retained because it is more stable for the coupled pipeline (season, wind, leaf lifecycle) and preserves consistent branch indexing for downstream state maps.

For emitted segments, geometry follows depth and stem-progress tapering:

$$L = L_0 \lambda_d^d \lambda_s^m \xi_L, \quad R = R_0 \rho_d^d \rho_s^m, \quad (3)$$

where d is branch depth, m is stem-step index, and $\xi_L \in [0.85, 1.15]$ in the implementation. Randomization is also injected into branch

count and orientation, e.g.

$$n_c = \text{clamp}(n_0 + \Delta n, 2, 6), \quad \Delta n \in \{-1, 0, +1\}, \quad (4)$$

and

$$\theta = \theta_0 (0.85 + 0.30 \xi_\theta), \quad \phi = \frac{2\pi i}{n_c} + \phi_0 + \epsilon_\phi, \quad (5)$$

so siblings do not remain coplanar or mechanically symmetric.

Directional guidance follows the colonization intuition of Runions et al. [2]. For a branch tip at $\mathbf{p}$, attractor $\mathbf{a}_i$, current direction $\mathbf{d}$, and occupancy o_i , the following quantities are defined:

$$\mathbf{n}_i = \frac{\mathbf{a}_i - \mathbf{p}}{\|\mathbf{a}_i - \mathbf{p}\|}, \quad d_i = \|\mathbf{a}_i - \mathbf{p}\|. \quad (6)$$

Within influence radius R_{inf} , the weights are

$$w_{\text{dist},i} = \left(1 - \frac{d_i^2}{R_{\text{inf}}^2}\right)^2, \quad (7)$$

$$w_{\text{front},i} = 0.30 + \max(0, \mathbf{n}_i \cdot \mathbf{d}),$$

$$w_{\text{occ},i} = \frac{1}{1 + 0.7 o_i},$$

$$w_i = w_{\text{dist},i} w_{\text{front},i} w_{\text{occ},i}, \quad \mathbf{d}_{\text{col}} = \text{normalize}\left(\frac{\sum_i w_i \mathbf{n}_i}{\sum_i w_i}\right). \quad (8)$$

The corrected growth direction is modeled as a sequential convex combination of three directional priors: colonization guidance, apical tendency, and tropism. Each stage uses a coefficient in $[0, 1]$ and is renormalized:

$$\mathbf{d}_1 = \text{normalize}((1 - \beta)\mathbf{d} + \beta\mathbf{d}_{\text{col}}), \quad (9)$$

$$\mathbf{d}_2 = \text{normalize}((1 - \alpha)\mathbf{d}_1 + \alpha\mathbf{d}_{\text{apical}}), \quad (10)$$

$$\mathbf{d}_3 = \text{normalize}((1 - \tau)\mathbf{d}_2 + \tau\mathbf{d}_{\text{tropism}}). \quad (11)$$

$$\beta = s_c \cdot \text{clamp}\left(\frac{\ell}{5}, 0.30, 1.0\right), \quad (12)$$

$$\alpha = 0.12 a_d \cdot \text{clamp}(1 - 0.12\ell, 0.20, 1.0),$$

$$\tau = t_s \cdot \text{clamp}(0.35 + 0.15\ell, 0.35, 1.0),$$

where s_c , a_d , and t_s correspond to canopy-guidance strength, apical-dominance strength, and tropism strength, respectively, and ℓ denotes a local developmental-level proxy used to modulate these weights.

Elm-oriented canopy assumptions were adopted in these priors: a broad lifted crown, reduced low attractor density, and depth-dependent outward spread. Concretely, attractors are sampled in an ellipsoidal envelope with inner-core exclusion, and branch directions are biased outward with an upward term that decays with depth. This produces the “trunk first, lateral crown later” behavior expected for an elm-like silhouette.

Finally, topology growth is bounded (`maxBranches`) to keep meshing, leaves, and simulation costs stable in real time. Branch geometry is meshed as tapered cylinders after stage and guidance updates, with normals recomputed at each rebuild.

4.3 Leaf Placement and Atlas Mapping

Leaves are generated only on active branches, i.e., branches whose stage has been unlocked by L-system expansion and whose age exceeds the leaf-emission threshold. Inactive branches are kept leaf-free, which avoids early canopy clutter and keeps juvenile stages visually plausible.

For active branches, a cylindrical phyllotaxis parameterization is used: each candidate leaf index i is mapped to one axial coordinate and one azimuth around the branch axis. This follows classical phyllotaxis observations and their procedural adaptation in plant modeling [1, 3, 13, 14]. The key idea is to use a constant divergence equal to the golden angle so that successive insertions avoid short-period angular repetition. For a branch with endpoints $\mathbf{p}_0, \mathbf{p}_1$, direction $\mathbf{d}$, radius r_b , and local transverse frame $(\mathbf{r}, \mathbf{f})$, the following relations are used:

$$s_i = \frac{i + 0.5}{N_b}, \quad t_i = \text{clamp}(0.20 + 0.72 s_i + \epsilon_i, 0.15, 0.95), \quad (13)$$

$$\theta_i = \theta_0 + i \varphi_g + \epsilon_{\theta,i}, \quad \varphi_g = 2\pi \left(1 - \frac{1}{\Phi}\right) \approx 137.507764^\circ, \quad \Phi = \frac{1 + \sqrt{5}}{2}, \quad (14)$$

where Φ is the golden ratio. Because $\varphi_g/2\pi$ is irrational, finite prefixes of $\{\theta_i\}$ fill the azimuth quasi-uniformly instead of aligning into a few preferred directions. In practice, this reduces clumping and visible striping before overlap correction.

$$\mathbf{u}_i = \text{normalize}(\mathbf{r} \cos \theta_i + \mathbf{f} \sin \theta_i), \quad \rho_i = r_b(0.65 + 0.20 \xi_i) + 0.14 s_i^{\text{leaf}}, \quad (15)$$

$$\widehat{\mathbf{p}}_i = \mathbf{p}_0 + t_i \|\mathbf{p}_1 - \mathbf{p}_0\| \mathbf{d} + \rho_i \mathbf{u}_i. \quad (16)$$

Here, $\widehat{\mathbf{p}}_i$ is the pre-overlap candidate location. A second tip-cluster pass places additional leaves near $\mathbf{p}_1$ with small backward offsets to increase crown fullness at branch ends.

Leaf size couples branch age and seasonal state:

$$s_i^{\text{leaf}} = s_{0,i} g_{\text{age}} g_{\text{season}} g_{\text{old}} g_{\text{attach}}, \quad (17)$$

where $g_{\text{age}} = \text{clamp}(a_b/1.6, 0.12, 1)$ (branch-age growth factor), g_{season} is the season size scale, $g_{\text{old}} = 0.92 + 0.14 o_i$ uses deterministic leaf oldness $o_i \in [0, 1]$, and g_{attach} is attachment-dependent. Candidate activation is determined by a leaf-specific target attachment:

$$a_i^* = S\left(\frac{\rho_{\text{eff}} - \eta_i + \delta}{2\delta}\right), \quad S(x) = x^2(3 - 2x), \quad x \in [0, 1], \quad (18)$$

where η_i is deterministic per-leaf noise, δ is the transition band, and ρ_{eff} is the effective seasonal density (including age-order shedding in Fall/Winter). In Spring, new buds are intentionally initialized very small and then increased continuously by attachment filtering:

$$a_i^{(t)} = (1 - \kappa_i) a_i^{(t-1)} + \kappa_i a_i^*, \quad g_{\text{attach}} = \text{mix}(0.02, 1.0, a_i^{(t)}), \quad (19)$$

with

$$\kappa_i = \begin{cases} (0.055 + 0.055 \xi_i) q_{\text{spring}}, & a_i^* > a_i^{(t-1)}, \\ 0.010 + 0.040 \xi_i, & a_i^* \leq a_i^{(t-1)} \text{ and shedding enabled,} \\ 0.060 + 0.080 \xi_i, & a_i^* \leq a_i^{(t-1)} \text{ otherwise,} \end{cases} \quad (20)$$

where $\xi_i \in [0, 1]$ is deterministic noise and $q_{\text{spring}} < 1$ slows early spring rise for visible bud growth. So leaves emerge from small sizes and grow gradually instead of popping to mature size. A small mandatory set of body and tip leaves is also preserved on active branches to avoid visually empty branches at medium seasonal densities.

To prevent leaf intersections, each candidate is approximated by a sphere $(\mathbf{p}_i, r_i)$ with $r_i = 0.45 s_i^{\text{leaf}}$, and soft repulsion is applied against previously placed spheres $(\mathbf{c}_j, r_j)$. With separation factor

$k_{\text{sep}} = 1.10$, overlap set $\mathcal{N}_i = \{j : \|\mathbf{p}_i - \mathbf{c}_j\| < k_{\text{sep}}(r_i + r_j)\}$, and relaxation $\eta = 0.7$, one iteration is

$$\Delta \mathbf{p}_i = \sum_{j \in \mathcal{N}_i} \mathbf{n}_{ij} (k_{\text{sep}}(r_i + r_j) - d_{ij}), \quad \mathbf{p}_i \leftarrow \mathbf{p}_i + \eta \frac{\Delta \mathbf{p}_i}{\max(1, |\mathcal{N}_i|)}, \quad (21)$$

with $d_{ij} = \|\mathbf{p}_i - \mathbf{c}_j\|$ and $\mathbf{n}_{ij} = (\mathbf{p}_i - \mathbf{c}_j)/d_{ij}$ (or a radial fallback direction when $d_{ij} \approx 0$). Three iterations are run per candidate, and an additional minimum radial distance from the branch axis is enforced to keep leaves outside the branch core.

To reduce temporal pumping during rebuilds, final positions are filtered by deterministic leaf ID:

$$\mathbf{p}_i^t = (1 - \lambda_i) \mathbf{p}_i^{t-1} + \lambda_i \widehat{\mathbf{p}}_i^t. \quad (22)$$

The smoothing gain is $\lambda_i = \text{mix}(\lambda_0, 0.03, \sigma)$, where λ_0 is the base position-smoothing parameter and $\sigma \in [0, 1]$ is the seasonal stability control.

For appearance, 9 distinct Spring/Summer leaf variants and 4 Fall variants are used. The seasonal appearance change is continuous, not discrete. Let $k_s \in \{0, \dots, 8\}$ be the spring/summer variant index and $k_f = M(k_s) \in \{0, \dots, 3\}$ the mapped fall variant (implemented as a fixed lookup $M = [0, 1, 2, 1, 0, 2, 3, 0, 2]$). With seasonal blend $b \in [0, 1]$, leaf albedo and opacity are blended as

$$\mathbf{C}(u, v) = (1 - b) \mathbf{C}_{k_s}^{\text{green}}(u, v) + b \widetilde{\mathbf{C}}_{k_f}^{\text{fall}}(u, v), \quad (23)$$

$$A(u, v) = ((1 - b) A_{k_s}^{\text{green}}(u, v) + b A_{k_f}^{\text{fall}}(u, v)) \alpha_s, \quad (24)$$

where α_s is a season-dependent alpha scale. During transitions, b is driven by the same smooth quintic scheduler used by the season controller.

Per-leaf color diversity in Fall is controlled by a deterministic scalar $v_i \in [0, 1]$ derived from leaf identity. This value drives a yellow/gold/orange/brown palette with limited muted-red contribution:

$$\widetilde{\mathbf{C}}_i^{\text{fall}} = \text{mix}(\mathbf{C}_i^{\text{fall}}, \mathbf{C}_i^{\text{fall}} \odot \mathbf{H}(v_i), 0.68), \quad (25)$$

where $\odot$ denotes component-wise multiplication and $\mathbf{H}(v)$ is

$$\mathbf{H}(v) = J(v) \text{ mix}(\mathbf{T}(v), \mathbf{c}_r, r(v)), \quad (26)$$

with piecewise base tone

$$\mathbf{T}(v) = \begin{cases} \text{mix}(\mathbf{c}_y, \mathbf{c}_g, v/0.45), & v < 0.45, \\ \text{mix}(\mathbf{c}_g, \mathbf{c}_o, (v - 0.45)/0.37), & 0.45 \leq v < 0.82, \\ \text{mix}(\mathbf{c}_o, \mathbf{c}_b, (v - 0.82)/0.18), & v \geq 0.82, \end{cases} \quad (27)$$

and muted-red weight/value jitter

$$r(v) = 0.22 S\left(\frac{v - 0.68}{0.27}\right), \quad J(v) = 0.92 + 0.14 \text{ fract}(13.37 v). \quad (28)$$

Color anchors are $\mathbf{c}_y = (1.10, 1.05, 0.76)$, $\mathbf{c}_g = (1.05, 0.93, 0.62)$, $\mathbf{c}_o = (0.98, 0.78, 0.50)$, $\mathbf{c}_b = (0.78, 0.62, 0.44)$, and $\mathbf{c}_r = (0.90, 0.60, 0.50)$. Because v_i is stable for each leaf key, canopy variegation is temporally stable and does not flicker frame to frame.

4.4 Season Controller

The seasonal module interpolates between parameter sets:

- **Spring**: full density, smaller fresh leaves, green tint.
- **Summer**: full density and full size.
- **Fall**: reduced density and size, fall texture blend.
- **Winter**: near-leafless canopy with muted tint.

Transitions use continuous easing functions, specifically cubic smoothstep $S(x) = x^2(3 - 2x)$ and quintic smootherstep $Q(x) = x^3(10 - 15x + 6x^2)$, with scheduled mesh rebuilds so parameter changes remain visually continuous in time.

The Winter $\rightarrow$ Spring transition uses a gated regrowth ramp so new leaves emerge only after old-state cleanup has progressed sufficiently:

$$\begin{aligned} r_{\text{spring}} &= \text{smoothstep}(0.84, 1.0, t), \\ s_{\text{leaf}} &= \text{mix}(0.06, s_{\text{spring}}, r_{\text{spring}}^2). \end{aligned} \quad (29)$$

This prevents old leaves from morphing into new ones and keeps spring emergence visibly gradual.

4.5 Leaf Detachment and Falling Dynamics

When branch leaves are no longer active, each leaf key spawns a falling leaf object with inherited visual properties (size, variant, color variation, and detached orientation basis). This preserves visual identity between attached and detached states.

Airborne state is explicitly represented by a binary condition (landed = false). While airborne, motion is integrated with substeps ($\Delta t \leq 1/90$ s) for contact robustness:

$$\mathbf{v}_{n+1} = \frac{\mathbf{v}_n + (\mathbf{f}_{\text{wind}} - g\mathbf{y}) \Delta t}{1 + c_d \Delta t}, \quad \mathbf{p}_{n+1} = \mathbf{p}_n + \mathbf{v}_{n+1} \Delta t, \quad (30)$$

with vertical speed clamped to terminal velocity. Horizontal wind forcing follows a phase-driven field:

$$\mathbf{f}_{\text{wind}} = k_w [\cos \phi, 0, \sin(1.13\phi)]^\top, \quad (31)$$

$$\phi = \phi_0 + t(0.9 + 0.8 s_{\text{drop}}), \quad (32)$$

$$k_w = 0.22 (0.35 + 1.25 w_{\text{blend}}). \quad (33)$$

Airborne orientation is derived from horizontal travel direction and a damped flap term. Let $\mathbf{u}_h$ be normalized horizontal velocity (or a yaw fallback when speed is very low), then

$$\mathbf{r}_{\text{air}} = \text{normalize}([-u_{h,z}, 0, u_{h,x}]^\top), \quad (34)$$

$$\mathbf{u}_{\text{air}} = \text{normalize}(\mathbf{u}_h + [0, f_{\text{flap}}, 0]^\top), \quad (35)$$

where f_{flap} is attenuated by both horizontal speed and ground proximity to avoid near-ground oscillation artifacts. The displayed orientation is a smooth blend from the detached branch pose to this aerodynamic pose:

$$(\mathbf{r}, \mathbf{u}) = \text{mix}((\mathbf{r}_{\text{det}}, \mathbf{u}_{\text{det}}), (\mathbf{r}_{\text{air}}, \mathbf{u}_{\text{air}}), \tau_o), \quad (36)$$

with τ_o increasing over a short detach-orientation window.

Ground contact is not tested at the leaf center; it is detected from the lowest oriented quad corner. If that corner crosses the ground between two substeps, a swept interpolation estimates the touchdown instant. The corresponding support corner index is stored, landed is set to true, and a settling phase then damps residual lateral velocity while preserving most touchdown tilt with only limited flattening. Ground litter persists with season-dependent aging and eventually shrinks/fades.

To avoid losing detached leaves when the fallen-leaf pool is saturated, capacity handling evicts the oldest landed leaf first, so new detach events are not silently dropped.

4.6 Wind Simulation

Wind is toggled at runtime and blended in/out smoothly through a scalar `windBlend`. The effective wind field has:

- slowly drifting dominant direction,
- independent gust envelope dynamics,
- canopy height and radial profile weighting.

The dominant horizontal wind direction is not prescribed as a fixed sinusoid. Instead, a low-frequency stochastic target $\mathbf{d}^* \in \mathbb{R}^2$ is tracked by a second-order damped system:

$$\ddot{\mathbf{d}} + 2\zeta_d \omega_d \dot{\mathbf{d}} + \omega_d^2 \mathbf{d} = \omega_d^2 \mathbf{d}^*, \quad (37)$$

followed by normalization $\mathbf{d} \leftarrow \mathbf{d}/\|\mathbf{d}\|$. The gust envelope g is modeled by the same dynamical form with its own target g^* :

$$\ddot{g} + 2\zeta_g \omega_g \dot{g} + \omega_g^2 g = \omega_g^2 g^*. \quad (38)$$

This decoupling removes regular phase locking while keeping temporal continuity in both direction and amplitude.

Branch node offsets are then simulated as damped oscillators toward wind targets:

$$\ddot{\mathbf{x}} + 2\zeta \omega \dot{\mathbf{x}} + \omega^2 \mathbf{x} = \omega^2 \mathbf{x}_{\text{target}}. \quad (39)$$

This realizes branch bending with a physically guided reduced-order response, following the same modeling direction as prior real-time wind studies while keeping the implementation lightweight for interactive use [4, 5]. Flexibility is depth- and radius-dependent:

$$\begin{aligned} r_n &= \text{clamp}(r/r_{\text{ref}}, 0, 1), \quad f_r = 1 - r_n^{0.45}, \quad f_d = \text{smoothstep}(0, 5.5, d), \\ f_{\text{branch}} &= \text{clamp}(0.08 + 0.62 f_d + 0.30 f_r + b_{\text{tip}}, 0.05, 1.20), \end{aligned} \quad (40)$$

where $b_{\text{tip}} = 0.12$ for terminal branches and 0 otherwise. Consequently, twigs and upper canopy respond strongly, while trunk and lower canopy remain comparatively calm.

Leaf motion under wind combines:

- inherited branch endpoint motion,
- local flutter through small angle modulation,
- per-leaf phase variation to prevent synchronous motion.

4.7 Rendering and Shadows

Rendering uses two passes:

- (1) **Shadow pass**: depth maps rendered from each light.
- (2) **Main pass**: textured shading with shadow lookup.

For each light, the light-space transform is

$$\mathbf{M}_{\text{depth}} = \mathbf{P}_{\text{ortho}} \mathbf{V}_{\text{light}}, \quad (41)$$

with $\mathbf{P}_{\text{ortho}} = \text{ortho}(-s, s, -s, s, 0.1, 3R)$ and $\mathbf{V}_{\text{light}} = \text{lookAt}(\mathbf{p}_\ell, \mathbf{c}_{\text{scene}}, \mathbf{y})$. Three depth maps are allocated, each at 2000×2000 .

In the main pass, the per-light shadow test uses a constant bias $b = 0.005$:

$$S_i = \begin{cases} 0, & z_{\text{curr}} - b > z_{\text{shadow}}, \\ 1, & \text{otherwise.} \end{cases} \quad (42)$$

Shading is Lambertian with ambient support:

$$\mathbf{L}_{\text{out}} = 0.06 \mathbf{a} + \sum_i (S_i \mathbf{L}_i \mathbf{a} \max(\mathbf{n} \cdot \mathbf{w}_i, 0)), \quad (43)$$

where $\mathbf{a}$ is the selected albedo (bark, wall texture, or season-blended leaf atlas sample), and the ambient term is applied once per fragment.

Algorithm 2 Main Frame Update (Simplified)

```

1: Read input and set toggles (growth/season/wind/restart)
2: Update season transition scalar and target parameters
3: Update growth stage and branch activation status
4: Update wind field (direction, gust, blend, profile)
5: For each active canopy leaf key:
6:   compute attachment, orientation, size, and color state
7: Detect removed leaf keys and spawn falling-leaf entries with
   inherited state
8: Integrate falling leaves with substeps and contact handling
9: if rebuild cooldown elapsed or transition requires then
10:   Regenerate branch mesh and leaf mesh
11: Render depth maps for shadows
12: Render branches, canopy leaves, and fallen leaves

```

To keep foliage silhouette consistent between depth and color passes, leaf alpha testing uses the same threshold in both shaders:

$$\text{discard if } \alpha < 0.42. \quad (44)$$

This eliminates square-card shadows and depth/color mismatch. Bark and leaves use separate material branches and texture slots; fallen leaves are rendered with season-aware blending but can keep old-fall appearance while spring cleanup is active.

4.8 Per-Frame Update Pipeline

The final implementation uses a fixed per-frame update order to preserve cross-module coherence:

- (1) process user input (G, C, W, R),
- (2) advance season state and transition progress,
- (3) update growth and branch activation state,
- (4) update wind state and branch/leaf deformation targets,
- (5) spawn and integrate detached/falling leaves,
- (6) rebuild meshes if cooldown and transition conditions require it,
- (7) execute shadow pass and main rendering pass.

Order is important: if detached leaves are updated before branch-state changes are finalized, identity continuity can break (leaves removed from canopy may not enter the falling pool in the same frame). Likewise, wind updates must be synchronized with the active deformation path to avoid double movement.

4.9 Parameterization Strategy

The implementation uses explicit parameter groups rather than hidden constants, enabling controlled tuning:

- **Growth parameters:** grammar iterations, angle, length decay, branch/leaf caps.
- **Season parameters:** density, scale, tint, texture blend, transition duration.
- **Wind parameters:** blend response, direction/gust filtering, canopy profile strength.
- **Falling-leaf parameters:** drag, gravity, terminal speed, landing settling, ground aging.

Table 3. Key parameter groups and their visual impact.

Parameter group	Example controls	Dominant visual effect
Growth	L-system iterations, branching angle, length decay, max branches/leaves	Crown topology, branch density, global tree silhouette
Season	Transition duration, leaf density/size scales, tint, atlas blend	Per-season look, smoothness of transitions, canopy fullness
Wind	Blend rate, gust frequency/amplitude, canopy-height weighting	Degree/nature of sway, tip emphasis, motion realism
Leaf lifecycle	Detach thresholds, drag/gravity, landing damping, ground aging	Falling plausibility, litter persistence, transition cleanliness
Rendering	Shadow resolution, bias, alpha cutoff consistency	Shadow stability, card-artifact suppression, overall readability

5 Modifications & Improvements

This section summarizes representative corrections made while integrating geometry, simulation, and rendering in one real-time vegetation pipeline. The focus is on common failure modes and practical remedies that are likely to appear in similar implementations.

5.1 Common Challenges and Corrections

The principal challenges and corresponding fixes are summarized in the following table.

5.2 Temporal Stability in Canopy Updates

A recurrent artifact in animated foliage is temporal popping when canopy occupancy and leaf size are updated discretely. The final version uses:

- attachment smoothing per leaf key to avoid branch-level binary changes,
- minimum attachment floor to keep tiny transitional leaves visible while growing,
- deterministic identity persistence across mesh rebuilds,
- season-aware rebuild cadence to prevent low-frequency visual stepping.

Together these controls make growth, maturation, and removal visually continuous.

5.3 Shadow Stability and Z-Buffer Artifacts

Thin foliage and fine bark detail are sensitive to depth precision, so early shadowed renders can exhibit light flicker when bias and depth tests are not tuned. Following standard shadow-mapping practice [11], this issue was addressed by stabilizing light-space projection bounds and applying a calibrated depth bias ($b = 0.005$).

Identical alpha cutout logic was also enforced in depth and shading passes ($\alpha < 0.42$ discard), which removed depth/color mismatch artifacts on leaf cards [12].

5.4 Fall Texture Mapping and Color Diversity

The fall atlas contains four variants in a non-uniform layout (three top leaves and one lower cluster), so direct 2D tile indexing produced incorrect UV regions and torn silhouettes. UV remapping to explicit rectangles solved this. In addition, per-leaf variation values were introduced to distribute canopy color across yellow, orange, brown, and muted red tones. This removed the unrealistic “single-color canopy” look and better matched observed autumn heterogeneity.

5.5 Leaf Falling Realism

Detached leaves initially dropped too synchronously and sometimes vanished from branches without corresponding airborne leaves. Improvements included:

- deterministic leaf identity transfer to falling state (size, variant, color, basis),
- delayed orientation blending from attached pose to tumble motion,
- stable landing based on lowest-corner contact and post-contact settling,
- lifespan/aging control that keeps fall litter longer but clears rapidly before spring regrowth,
- capacity management that evicts oldest landed leaves first to preserve new detach events.

These changes made the full leaf cycle legible: grow → mature → recolor → detach → settle → disappear.

5.6 Wind Realism and Stability

Early wind versions were unrealistic for three concrete reasons. First, a single global sinusoidal forcing term produced nearly identical phase across trunk, branches, and leaves, so the canopy moved too coherently. Second, increasing only leaf flutter improved visual activity but weakened attachment perception because leaf motion diverged from branch motion. Third, branch-only deformation reduced this mismatch but made foliage appear comparatively static.

The final model combines these lessons: branch motion is driven by filtered stochastic direction and gust states (second-order damped dynamics), branch flexibility is weighted by depth and radius, and leaves inherit branch endpoint motion before adding small local angular flutter with per-leaf phase offsets. This separation of global and local components avoids rigid synchronization, preserves attachment cues, and yields a more natural hierarchical response under wind.

6 Results

6.1 Integrated Runtime Application

The primary result is a single real-time application in which geometry generation, simulation, and rendering run as one coupled system. Rather than presenting isolated demonstrations, the implementation supports interactive runtime control over growth, season progression, wind, mesh refinement, and capture utilities. This integration is the central project outcome because it validates that the three required modules operate coherently under continuous



Fig. 3. Leaf detachment and falling phase during seasonal transition.

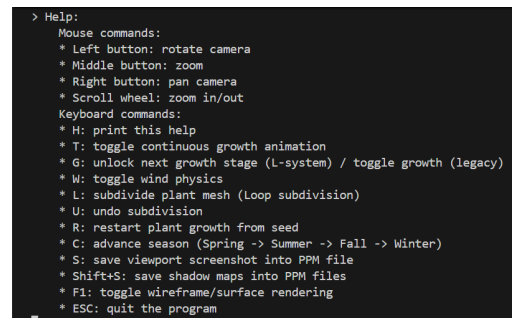


Fig. 4. Help-command view of the application, showing implemented run-time operations and interactions (summarized in Table 5).

state changes. Figure 1 summarizes the seasonal output at a glance, Figure 2 shows staged growth under repeated G input, and Figure 4 documents the runtime interaction interface.

6.2 Seasonal Progression and Leaf Lifecycle

The seasonal simulation produces a full annual loop while preserving branch structure. In Spring, new leaves emerge at small size and gradually expand. Summer reaches maximal canopy density with mature green foliage. Fall combines gradual chromatic shift with staged detachment, and Winter produces a sparse canopy with decaying ground litter. The transition policy avoids direct old-to-new leaf replacement: remaining old leaves are detached and cleared before new Spring growth begins.

An important behavioral result is lifecycle continuity. A canopy leaf that detaches is transferred into the airborne simulation with inherited state (size, orientation, variant, and color context), then transitions to landed litter and finally fades. This yields consistent visual causality from branch to ground, illustrated by Figure 3. The same lifecycle is also documented in the accompanying project video.

6.3 Wind Response

Wind response is strongest near upper canopy and branch tips, while trunk and lower regions remain comparatively stable. This spatial gradient produces a more plausible tree motion profile than uniform global sway. Under gusts, branches show delayed response

and settling rather than immediate periodic reversal, and attached leaves inherit branch deformation before adding local flutter. The resulting motion reads as coupled, not independent, which preserves the visual impression that leaves remain physically attached to moving branches.

6.4 Robustness Under Coupled Dynamics

When growth, seasonal transition, and wind are active together, the final system remains visually stable and responsive. Compared with early versions, the integrated pipeline substantially reduces temporal artifacts (popping, flashing, and pumping) during state changes. The final output therefore satisfies two criteria simultaneously: interactive controllability (Table 5) and continuity of motion/appearance across long seasonal and short-term wind dynamics.

7 Evaluation

7.1 Validation Process

For this course project, evaluation was performed through controlled iterative validation rather than a full benchmark campaign. The process kept camera and interaction conditions fixed, replayed the same growth/season/wind sequences, and compared early and final outputs using screenshots and video captures. This enabled verification of temporal continuity, motion plausibility, and runtime stability under the same operating conditions.

7.2 Observed Improvements

The final implementation shows clear improvement over the initial baseline in continuity, realism, and interaction robustness.

8 Observations & Future Work

8.1 Limitations

Current limitations are primarily related to elm-specific biological realism and model fidelity:

- **Elm lifecycle simplification:** the seasonal controller follows a stylized four-season schedule and does not yet model elm phenology in detail (budburst timing variability, progressive senescence patterns, and climate-dependent leaf retention/drop behavior).
- **Elm growth-pattern simplification:** branch emergence, branching angles, internode spacing, taper, and crown development are driven by procedural heuristics rather than calibrated elm allometry and age-dependent architectural rules.
- **Reduced biological adaptation:** long-term responses such as stress-driven crown asymmetry, pruning/recovery behavior, and inter-annual growth adaptation are not represented.
- **Rendering/compute constraints:** the pipeline remains raster-first and CPU-heavy for several update stages, which limits both physical-light realism and large-scale simulation complexity.

8.2 Planned Extensions

The next milestones are:

- (1) **More trees:** add multi-tree placement with species/seed variation.

- (2) **L-system refinement:** improve botanical plausibility via parameter fitting and pruning rules.
- (3) **Advanced wind:** combine branch modal response with local turbulence volumes.
- (4) **Ray tracing:** hybrid or full ray-traced shadows/reflections for foliage.

Additional technical improvements:

- GPU instancing for leaves and fallen leaves,
- level-of-detail strategy for far trees,
- data logging and quantitative evaluation (frame time, leaf counts, rebuild cost).

9 Conclusion

This project demonstrates a complete interactive tree pipeline where procedural growth, seasonal state evolution, wind deformation, and leaf life-cycle simulation are integrated into one consistent system. The main contribution is not a single isolated algorithm, but a robust coupling strategy that preserves temporal continuity while supporting rich artistic controls.

In the final version, the tree exhibits smoother seasonal transitions, coherent detachment-to-fall behavior, improved fall atlas usage, and more plausible canopy-weighted wind response. Shadow rendering and material blending remain stable across all seasonal states, including fallen leaf layers on the ground.

The implementation forms a strong foundation for extending to larger scenes and more physically detailed vegetation models. With the proposed future work (multi-tree support, enhanced wind, and ray-traced effects), the same architecture can evolve into a higher-fidelity outdoor rendering framework.

References

- [1] Prusinkiewicz, P., and Lindenmayer, A. 1990. *The Algorithmic Beauty of Plants*. Springer-Verlag.
- [2] Runions, A., Lane, B., and Prusinkiewicz, P. 2007. Modeling trees with a space colonization algorithm. *Eurographics Workshop on Natural Phenomena (NPH'07)*, 63–70. <https://doi.org/10.2312/NPH/NPH07/063-070>.
- [3] Palubicki, W., Horel, K., Longay, S., Runions, A., Lane, B., Mech, R., and Prusinkiewicz, P. 2009. Self-organizing tree models for image synthesis. *ACM Transactions on Graphics* 28, 3 (SIGGRAPH 2009), Article 58. <https://doi.org/10.1145/1531326.1531364>.
- [4] Derzaph, T. L. M., and Hamilton, H. J. 2013. Effects of Wind on Virtual Plants in Animation. *International Journal of Computer Games Technology* 2013, Article 674848.
- [5] Habel, R., Kusternig, A., and Wimmer, M. 2009. Physically Guided Animation of Trees. *Computer Graphics Forum* 28, 2.
- [6] Stava, O., Benes, B., Mech, R., Aliaga, D. G., and Kristof, P. 2010. Inverse Procedural Modeling by Automatic Generation of L-systems. *Computer Graphics Forum* 29, 2.
- [7] Stava, O., Pirk, S., Kratt, J., Chen, B., Mech, R., Deussen, O., and Benes, B. 2014. Inverse Procedural Modelling of Trees. *Computer Graphics Forum*.
- [8] Guo, J., Jiang, H., Benes, B., Deussen, O., Zhang, X., Lischinski, D., and Huang, H. 2020. Inverse Procedural Modeling of Branching Structures by Inferring L-Systems. *ACM Transactions on Graphics* 39, 5.
- [9] Lee, J.-J., Li, B., and Benes, B. 2024. Latent L-systems: Transformer-based Tree Generator. *ACM Transactions on Graphics* 43, 1.
- [10] Yildiz, C. 2011. An Implementation on Real-time Animation of Trees Swaying in Wind Fields. Technical report, Bilkent University.
- [11] Williams, L. 1978. Casting curved shadows on curved surfaces. *Proceedings of SIGGRAPH*.
- [12] Reeves, W. T., and Blau, R. 1985. Approximate and probabilistic algorithms for shading and rendering structured particle systems. *Proceedings of SIGGRAPH*.
- [13] Vogel, H. 1979. A better way to construct the sunflower head. *Mathematical Biosciences* 44, 3–4, 179–189. [https://doi.org/10.1016/0025-5564\(79\)90080-4](https://doi.org/10.1016/0025-5564(79)90080-4).
- [14] Jean, R. V. 1994. *Phyllotaxis: A Systemic Study in Plant Morphogenesis*. Cambridge University Press.

Table 5. Implemented runtime interactions.

Input	Runtime action
Mouse Left	Rotate camera
Mouse Middle	Zoom camera
Mouse Right	Pan camera
Mouse Wheel	Zoom in/out
H	Print help and available controls
T	Toggle continuous growth animation (pause/resume global timer)
G	In L-system mode: unlock next growth stage; in legacy mode: toggle branch growth
C	Advance to next season
W	Toggle wind physics
L	Apply one Loop subdivision step to the plant mesh
U	Undo one subdivision step
R	Restart with a new random seed
S	Save viewport screenshot as PPM
Shift+S	Save shadow-map depth images as PPM
F1	Toggle wireframe/surface rendering
ESC	Quit application

Table 6. Summary of observed improvements from early baseline to final integrated behavior.

Category	Early behavior	Final behavior
Season transitions	Frequent flashing/popping; old/new leaves mixed	Staged transitions with explicit old-leaf flush and delayed regrowth
Fall appearance	Uniform reddish canopy; atlas tearing artifacts	Per-leaf hue variation and explicit non-grid atlas UVs
Detachment continuity	Leaves vanish from branch without matching airborne leaf	One-to-one identity transfer from attached to falling state
Ground interaction	Snap/jump when touching ground	Swept contact, substepping, and damped settling
Wind realism	Periodic synchronized canopy motion	Filtered gusts, hierarchical flexibility, and canopy-weighted response
Wind artifacts	Unintended branch fragments under wind in partial growth stages	Stable branch frames and visibility gating of inactive growth stages